

# LAB 2

## Email Client Application

---

Alexandria University, Faculty of Engineering

Spring 2026

|                 |                     |
|-----------------|---------------------|
| <b>Student:</b> | <b>Omar Roushdy</b> |
| <b>ID:</b>      | <b>9091</b>         |

## 1. Project Overview

This project implements a Python-based email client that can send emails using the SMTP protocol and receive emails using the IMAP protocol. A bonus GUI application with push notifications is also included.

The system consists of three files:

| File             | Protocol              | Description                                                                 |
|------------------|-----------------------|-----------------------------------------------------------------------------|
| send_email.py    | SMTP + TLS (port 587) | Sends emails via SMTP with TLS encryption.                                  |
| receive_email.py | IMAP + SSL (port 993) | Fetches the latest email from the inbox via IMAP.                           |
| email_gui.py     | Both SMTP and IMAP    | Bonus: GUI with input fields, send/receive buttons, and push notifications. |

## 2. Sending Emails (send\_email.py)

This script handles sending emails through an SMTP server. It builds the email using MIME, connects to the server, upgrades to TLS, authenticates, and sends the message.

### SMTP Flow Summary

| Step | SMTP Command | What Happens                                             |
|------|--------------|----------------------------------------------------------|
| 1    | TCP Connect  | Client opens a TCP socket to the server on port 587.     |
| 2    | EHLO         | Client greets the server and discovers its capabilities. |
| 3    | STARTTLS     | Connection is upgraded from plain text to encrypted TLS. |
| 4    | EHLO (again) | Required by protocol after switching to TLS.             |
| 5    | AUTH LOGIN   | Client sends credentials to authenticate.                |
| 6    | MAIL FROM    | Client tells the server who is sending the email.        |
| 7    | RCPT TO      | Client tells the server who should receive the email.    |
| 8    | DATA         | Client transmits the email headers and body.             |
| 9    | QUIT         | Client closes the SMTP session and TCP connection.       |

### 3. Receiving Emails (receive\_email.py)

This script fetches the most recent email from the user's inbox using the IMAP protocol over SSL. It connects, selects the inbox, downloads the latest message, and parses it.

#### 3.1 IMAP Flow Summary

| Step | IMAP Command      | What Happens                                                         |
|------|-------------------|----------------------------------------------------------------------|
| 1    | TCP + SSL Connect | Client opens an encrypted connection to the IMAP server on port 993. |
| 2    | LOGIN             | Client authenticates with email and password.                        |
| 3    | SELECT INBOX      | Client opens the inbox and gets the total number of emails.          |
| 4    | FETCH n (RFC822)  | Client downloads the full raw email by its ID number.                |
| 5    | Parse email       | Raw bytes are parsed into headers (From, Subject, Date) and body.    |
| 6    | CLOSE             | Deselects the mailbox.                                               |
| 7    | LOGOUT            | Ends the IMAP session and closes the connection.                     |

#### 3.2 Multipart vs Singlepart Emails

Emails come in two formats. Understanding this is important:

| Type       | Structure                                 | Example                                       |
|------------|-------------------------------------------|-----------------------------------------------|
| Singlepart | One body only (text/plain or text/html)   | A simple plain-text email with no attachments |
| Multipart  | Multiple parts: text + HTML + attachments | An email with images or file attachments      |

The screenshot shows a terminal window on the left displaying the raw IMAP and SMTP protocol messages for sending an email. The messages include commands like 'reply: b'250-CHUNKING\r\n', 'reply: b'250 SMTPUTF8\r\n', 'send: 'AUTH PLAIN AG9tYXIubS5kZXNpZ25lckBnbWFpbC5kZ29725==\r\n', and 'send: 'mail from:<omar.m.designer@gmail.com> size=...'. The terminal output ends with '[SMTP] Email sent successfully to omar.m.designer@gmail.com'.

On the right, there is a graphical user interface (GUI) for an email client. It has a 'Login Credentials' section with fields for 'Your Email' (omar.m.designer@gmail.com) and 'Password'. Below that is the 'Server Settings' section with 'SMTP Server' (smtp.gmail.com, Port: 587) and 'IMAP Server' (imap.gmail.com, Port: 993). The 'Compose Email' section has fields for 'To' (omar.m.designer@gmail.com), 'Subject' (my self testing), and 'Body' (this is somme teeeesting email). At the bottom, there are 'Send Email' and 'Receive Latest Email' buttons. A 'Success' dialog box is open, displaying 'Email sent successfully!' with an 'OK' button. The 'Output' section at the bottom shows the message '[SUCCESS] Email sent successfully!'.